

MoE Serving: Thesis Proposal

Plan A: Gate-Aware Error-Bounded Partial Combine

Plan B: Energy-Efficient MoE Serving under SLO

June 16, 2026

Gate-Aware Error-Bounded Partial Combine

Positioning

MoE inference is dominated by expert-parallel all-to-all communication. In a forward pass, both dispatch and combine are symmetric all-to-all stages, but combine is usually treated as a mandatory full-precision weighted aggregation step.

This plan asks whether the gate-weighted linear structure of combine can be used to reduce expensive second all-to-all traffic under bounded hidden-state perturbation.

Combine structure

$$y_t = \sum_{e \in S(t)} g_{t,e} \cdot o_{t,e}$$

The output is linear and recombinaible; gate weight $g(t,e)$ is already known before combine and naturally quantifies each expert contribution.

$$||\Delta y_t|| \leq \sum_{e \in \text{dropped}} g_{t,e} \cdot ||o_{t,e}||$$

Why Combine, Not Dispatch

Structural thesis

The combine phase is the only stage where gate-aware differential-precision transmission is structurally viable.

- Dispatch replicates the same hidden state to the top-k experts. Assigning different precision to replicas would require k independent quantization passes and disrupt collective regularity.
- Combine receives independent expert outputs $o_{t,e}$, each paired with a known gate weight $g_{t,e}$. Precision can be varied by contribution without changing routing semantics.

This difference is determined by the MoE computation graph, not by an engineering implementation trick.

Importance proxy

The simplest deployable proxy is rank inside the token's top-k set. Rank 1 has the highest gate weight; rank k is the lowest-ranked expert. Gate-bucket variants can be evaluated as an enhanced ablation.

Rank-LUT key: (layer_id, receiver_group, rank)

Claims and Validation

Claim C1: within-top-k contribution long tail

The key empirical question is whether $g_{t,e} \cdot ||o_{t,e}||$ has a pronounced long-tail distribution within the top-k set, not only across the global expert pool. Existing work supports expert-level imbalance, but does not directly prove this within-token property.

Validation: attach forward hooks to real MoE models such as DeepSeek-V2-Lite or Qwen-MoE, sample per-(token, expert) combine contributions, and plot their distributions by layer and rank.

Claim C2: routing drift as a quality-loss source

Quantization noise may perturb hidden states and change downstream router decisions. EAQuant's routing-fragility observation motivates this risk; the fraction of total loss should be measured directly.

Evaluation separates total loss from attribution: run approximated combine with free routing, then rerun with routing locked to the original model to estimate the contribution of routing drift.

Static Deployment Strategy

WHERE - receiver-port congestion

Assume the network core is not the bottleneck; contention appears at receiver ports. Receiver ports are clustered offline into hot, warm, and cold groups by aggregated expert traffic.

WHICH LAYER - sensitivity filtering

Approximate each layer in isolation and measure perplexity or accuracy impact. High-sensitivity layers are fixed to BF16; the optimizer only searches low-sensitivity layers.

HOW - Rank-LUT lookup

Runtime uses only a static table lookup. Precision choices include BF16, FP8, INT4, and drop. Dropping is reserved for the lowest-ranked expert position $R=k$, with gate renormalization as a best-effort scale correction.

$$precision = LUT[layer_id, receiver_group, rank]$$

Variables and Receiver Load

Decision variables

$$x_{l,r,R,p} \in \{0,1\}, \quad \sum_p x_{l,r,R,p} = 1$$

$x=1$ means all matching (token, expert) combine outputs use precision p for layer l , receiver group r , and rank R .

Bytes and receiver arrival rate

$$\begin{aligned} \text{bytes}(l,r,R) &= \sum_p \text{size}(p) \cdot x_{l,r,R,p} \\ \text{size}(\text{BF16}) &= 2B, \text{ size}(\text{FP8}) = 1B, \text{ size}(\text{INT4}) = 0.5B, \text{ size}(\text{drop}) = 0 \end{aligned}$$

$$\lambda_r(x) = \sum_{l,R,p} \text{size}(p) \cdot x_{l,r,R,p} \cdot \text{freq}(l,r,R) / T_{\text{step}}$$

$\text{freq}(l,r,R)$ is measured from offline traces: the number of pairs at layer l , receiver group r , and rank R .

Variable scale is $O(L_{\text{low}} \times |\text{groups}| \times k \times |P|)$. For example, 16 low-sensitivity layers $\times$ 3 groups $\times$ $k=2 \times 4$ precisions gives about 384 binary variables; even $k=6$ is about 1.1k.

WHICH-LAYER filtering fixes sensitive layers to BF16, so the MILP only covers low-sensitivity layers instead of all MoE layers.

Accuracy and TBT Constraints

Congestion objective

$$\text{minimize } U, \text{ s.t. } \lambda_r(x) / \mu_r \leq U \text{ for all } r$$

The objective is directly tied to receiver-port P99 utilization / queue length.

Accuracy constraint

$\delta_{l,R,p}$ is the marginal degradation when rank R in layer l switches to precision p ; $\delta=0$ for BF16.

$$\sum_{l,r,R,p} \delta_{l,R,p} \cdot x_{l,r,R,p} \cdot \text{freq}(l,r,R) / \sum_{l',r',R'} \text{freq}(l',r',R') \leq \epsilon$$

Receiver group affects accuracy only through frequency weights; high-sensitivity layers are fixed to BF16.

TBT constraint

$$\begin{aligned} TBT_{p99} = & T_{\text{attn}} + T_{\text{compute}}(\text{MoE FFN}) + T_{\text{dispatch}} \\ & + T_{\text{queue_combine}}(x) + T_{\text{other}}, \quad \text{with } U \leq \rho^* \end{aligned}$$

T_{other} covers layernorm, residual paths, sampling, and runtime overhead; $U \leq \rho^*$ bounds P99 queueing delay.

Oracle and Evaluation

Offline oracle upper bound

On a logged trace, allow each (token, expert) pair to choose its precision under a local combine-output MSE budget. This oracle is not used at runtime; it only measures how close the static Rank-LUT is to the theoretical upper bound.

$$b_{t,e}^* = \operatorname{argmin}_b \text{bytes}, \quad \text{s.t. } \|y^{\text{approx}}(t) - y^{\text{full}}(t)\|^2 \leq \delta$$

Baselines

- All-BF16 combine; uniform FP8/INT4 combine; rank-only heuristic; receiver-only heuristic; static Rank-LUT; offline oracle.

Decision criterion

The first experiment should test whether within-top-k contribution is long-tailed enough to justify dropping or low-precision transmission.

Energy-Efficient MoE Serving under SLO

Positioning

Existing MoE serving systems usually optimize latency and throughput. This plan builds an MoE-specific energy model and jointly optimizes expert placement and replication count under TBT/SLO constraints.

Decision surface

$$x_{l,e,g} \in \{0,1\}, \quad r_{l,e} = \sum_g x_{l,e,g}$$
$$\text{tier}(g,g') \in \{\text{NVLink}, \text{leaf/IB}, \text{spine}\}, \quad c_{\text{NVLink}} < c_{\text{leaf/IB}} < c_{\text{spine}}$$

Tradeoff

- Expert placement determines whether all-to-all traffic stays on NVLink, crosses leaf/IB, or reaches the spine.
- Co-locating frequently co-activated experts lowers communication energy and TBT, but may create hot GPUs and HBM pressure.
- Replicating hot experts reduces queueing and cross-node traffic, but adds static GPU power and HBM footprint.

Evidence target: a Pareto frontier over latency/TBT and J/token, sweeping placement, replica count, batch size, and topology tier.

Energy Model per Decode Step

Decomposition

$$\begin{aligned}
 E_{\text{step}} &= E_{\text{static}} + E_{\text{compute}} + E_{\text{comm}} \\
 E_{\text{static}} &= \sum_g [P_{\text{idle},g} + \rho_g \cdot (P_{\text{TDP},g} - P_{\text{idle},g})] \cdot T \\
 E_{\text{compute},l} &= \alpha_{\text{load},l} \cdot a_l + \beta_l \cdot n_{\text{tokens},l}, \quad a_l \in \{0,1\} \\
 E_{\text{comm}} &= \sum_l \sum_{(g,g')} D^l_{g \rightarrow g'}(x) \cdot c^{comm}_{g \rightarrow g'}
 \end{aligned}$$

Static and compute terms

- P_{idle} and P_{TDP} come from datasheets; for H100, a rough anchor is 70 W idle and 700 W TDP.
- α_{load} is computed from expert weight size, HBM bandwidth, and average power; β_l is derived from LLMCarbon's J/FLOP coefficient.
- a_l is a 0-1 replica-activation variable, avoiding weight-load charges for replica copies that receive no tokens.

Communication term

- $D^l_{g \rightarrow g'}(x)$ is decoupled using expert-pair co-activation frequencies from traces, with McCormick linearization.
- c_{comm} is expanded into NVLink / leaf / spine pJ/bit tiers; cross-rack traffic gives placement a clear topology-gradient signal.

Optimization Problem and Solving

Objective

$$\text{minimize}_x E_{\text{step}}(x) / B, \quad \text{subject to } TBT_{p99}(x) \leq SLO$$

The objective is J/token per decode batch, evaluated under the same TBT/SLO budget used for serving quality.

Constraints

$$T(x) \leq TBT_{99}^{SLO}, \quad \sum_g x_{l,e,g} \geq 1, \quad HBM_g(x) \leq Cap_g$$

$\rho_g \cdot T$ is conservatively linearized with $T = SLO$

$T(x)$ reuses Plan A's receiver-port queueing upper bound for the communication component T_g^{comm} . HBM constraints cap the total resident expert weights per GPU.

Solver path

- Small instances use MILP, e.g. Gurobi, to validate the model and produce interpretable placements.
- Larger deployments use Lagrangian relaxation or greedy heuristics, then report J/token, TBT, energy breakdown, and chosen replica counts.